

Advanced MySQL Topics

Maulana Hafidz Ismail

10 Oktober 2025

Contents

1	Functions and Triggers	1
1.1	Pengantar Functions dan Stored Procedures	1
1.2	Manfaat dan Organisasi Kode	1
1.3	Perbedaan Antara Functions dan Stored Procedures	1
1.4	Variabel dalam MySQL	1
1.5	Deklarasi Variabel	2
1.6	Mengatur Nilai Variabel	2
1.7	Contoh Penggunaan	2
1.8	Pembuatan Variabel di Stored Procedure	3
1.9	Parameter dalam MySQL	3
1.10	Contoh Penggunaan Parameter	3
1.11	Pengertian User-Defined Functions	4
1.12	Proses Pembuatan User-Defined Functions	4
1.13	Contoh Penggunaan	4
1.14	Fungsi yang Lebih Kompleks	5
1.15	Pengujian Fungsi	6
1.16	Menghapus Fungsi	6
1.17	Hubungan DETERMINISTIC dengan Memori dan Kinerja	6
1.18	Apa itu Delimiter?	6
1.19	Mengapa Mengubah Delimiter Penting?	7
1.20	Cara Mengubah Delimiter	7
1.21	Contoh Penggunaan DELIMITER	7
1.22	Membuat Stored Procedure	8
1.23	Perbedaan Antara Fungsi dan Prosedur	8
1.24	Apa itu MySQL Trigger	9
1.25	Cara Membuat dan Menghapus Trigger	9
1.26	Contoh Penggunaan Trigger	9
1.27	Manfaat Trigger	9
1.28	Jenis-Jenis Triggers	10
1.29	Klasifikasi Trigger	10
1.30	Contoh Trigger	10
1.31	Sintaks untuk Membuat Trigger	10
1.32	Contoh Penerapan Trigger	11
1.33	Membuat Trigger	11
1.34	Menghapus Trigger	11
1.35	Pengertian MySQL Scheduled Events	12
1.36	Sintaks untuk Membuat Scheduled Event	12
1.37	Menghapus Scheduled Event	13
1.38	MySQL Triggers dan Penggunaannya	13
1.39	Membuat dan Menghapus MySQL Triggers	13
1.40	Mengakses Kolom Tabel dalam Trigger	14
1.41	Keuntungan dan Kerugian dari Triggers	14

2	Database Optimization	15
2.1	Konsep Database Optimization	15
2.2	Jenis Pernyataan SQL	15
2.3	Teknik Optimasi untuk Data Retrieval Statemments	15
2.4	Teknik Optimasi untuk Data Change Statements	15
2.5	Pentingnya Optimasi Kueri SELECT	15
2.6	Pedoman Dasar untuk Optimasi SELECT	16
2.7	Contoh Praktis Optimasi SELECT	16
2.8	Mengoptimalkan Kueri SELECT Pada WHERE	16
2.9	Menggunakan Wildcard dan Indeks	17
2.10	Mengambil Data dari Tabel dengan Join	17
2.11	Pentingnya Optimasi Kueri SELECT	17
2.12	Teknik untuk Mengoptimalkan Kueri SELECT	17
2.13	Mengoptimalkan Kueri SELECT dengan Pernyataan Explain . .	18
2.14	Contoh Hasil Pernyataan EXPLAIN	18
2.15	Penjelasan Kolom dalam Hasil EXPLAIN	18
2.16	Rencana Kueri SELECT yang Lebih Efisien	20
2.17	Pengertian Index	21
2.18	Jenis-jenis Index	21
2.19	Operasi Query dan Indeks di MySQL	21
2.20	Proses Penggunaan Index	22
2.21	Definisi Transaksi	22
2.22	Pernyataan Transaksi dalam MySQL	22
2.23	Proses Pengelolaan Transaksi	23
2.24	Contoh Kasus Transaksi	23

1 Functions and Triggers

1.1 Pengantar Functions dan Stored Procedures

- Functions dan Stored Procedures memungkinkan Anda untuk menggunakan kembali kode dalam proyek database, sehingga tidak perlu menetik kode yang sama berulang kali.
- Contoh penggunaan adalah pada perusahaan Lucky Shrub yang menggunakan metode ini untuk memeriksa data stok di tabel produk mereka.
- A function in MySQL is also called a stored function.

1.2 Manfaat dan Organisasi Kode

- Tujuan utama dari functions dan stored procedures adalah untuk mengenkapsulasi kode, sehingga anda dapat memanggil blok kode untuk melakukan operasi tertentu dengan menggunakan nama pengidentifikasi.
- Keduanya membuat kode lebih konsisten, terorganisir, dan memperkenalkan reusability, yang memudahkan penggunaan dan pemeliharaan kode.

1.3 Perbedaan Antara Functions dan Stored Procedures

- Functions selalu mengembalikan nilai, sedangkan stored procedures tidak selalu mengembalikan nilai.
- Function hanya dapat memiliki input parameters, sedangkan stored procedures dapat memiliki input dan output parameters.
- Contoh sintaks untuk membuat stored procedure:

```
CREATE PROCEDURE procedure_name AS
BEGIN
    -- logic here
END;
```

- Contoh sintaks untuk menggunakan function:

```
SELECT MOD(x, y)
```

1.4 Variabel dalam MySQL

- Variabel adalah placeholder yang menyimpan nilai yang dapat berubah sesuai kebutuhan query. Variabel digunakan untuk mengoper nilai antara pernyataan SQL atau antara prosedur dan pernyataan SQL.

- Variabel dapat dibuat di dalam atau di luar stored procedure dan di dalam atau di luar pernyataan SELECT.
- Contoh sintaks untuk membuat variabel:

```
SET @variabel_name = value;
```

1.5 Deklarasi Variabel

- DECLARE digunakan untuk mendeklarasikan variabel lokal di dalam stored procedure. Variabel ini hanya dapat digunakan dalam konteks prosedur tersebut. Contoh:

```
DECLARE variable_name datatype;
```

- SET @ digunakan untuk mendapatkan variabel sesi (session variable) yang dapat diakses di seluruh sesi MySQL. Variabel ini diawali dengan simbol @. Contoh:

```
SET @variable_name = value;
```

1.6 Mengatur Nilai Variabel

1. SET digunakan untuk memberikann nilai kepada variabel yang telah dideklarasikan. Ini adalah cara yang umum untuk menginisialisasi variabel. Contoh:

```
SET variable_name = value;
```

2. Operator (:=) digunakan untuk menetapkan nilai ke variabel, baik untuk variabel lokal yang dideklarasikan dengan DECLARE maupun untuk variabel sesi yang diawali dengan @. Contoh:

```
variable_name := value;
```

1.7 Contoh Penggunaan

```
CREATE PROCEDURE example_procedure()
BEGIN
    DECLARE total_cost DECIMAL(10,2);
    SET total_cost = 100.00;
    SET @discount = 10;
    total_cost := total_cost - @discount;
END;
```

1.8 Pembuatan Variabel di Stored Procedure

- Menggunakan SET Command: Untuk menetapkan nilai pada variabel di dalam stored procedure, gunakan perintah SET. Contoh:

```
SET @order_id = 3;
```

- Menggunakan DECLARE Command: Untuk membuat variabel di dalam stored procedure, gunakan perintah DECLARE. Contoh:

```
DECLARE minimum_order_cost DECIMAL(10,2) DEFAULT 0;
```

1.9 Parameter dalam MySQL

- Parameter digunakan untuk mengoper argumen atau nilai ke fungsi atau prosedur dari luar. Ada tiga jenis parameter dalam stored procedures: IN, OUT, dan INOUT.
- Parameter IN: Parameter default yang digunakan untuk mengoper nilai ke dalam stored procedure. Contoh sintaks:

```
CREATE PROCEDURE procedure_name(IN parameter_name
                                datatype)

CREATE PROCEDURE calculate_tax(IN salary DECIMAL(10,2))
BEGIN
    DECLARE tax DECIMAL(10,2);
    SET tax = salary * 0.2;
    SELECT tax AS TaxAmount;
END;

CALL calculate_tax(5000);
```

1.10 Contoh Penggunaan Parameter

- Parameter OUT: Digunakan untuk mengoper nilai ke variabel di luar prosedur. Contoh:

```
CREATE PROCEDURE get_lowest_cost(OUT lowest_cost DECIMAL
                                (10,2))

CREATE PROCEDURE get_lowest_cost(OUT lowest_cost DECIMAL
                                (10,2))
BEGIN
    SELECT MIN(cost)
    INTO lowest_cost
    FROM orders;
```

```
END;

CALL get_lowest_cost(@lowest_cost);
SELECT @lowest_cost AS LowestCost;
```

- Parameter INOUT: Kombinasi dari IN dan OUT, digunakan untuk mengoper nilai dan mengembalikan nilai baru. Contoh:

```
CREATE PROCEDURE square_number(INOUT number INT)

CREATE PROCEDURE square_number(INOUT number INT)
BEGIN
    SET number = number * number;
END;

SET @num = 5;
CALL square_number(@num);
SELECT @num AS SquaredValue;
-- Output akan menjadi 25
```

1.11 Pengertian User-Defined Functions

- User-defined functions adalah fungsi yang dibuat untuk melakukan operasi yang tidak dapat diselesaikan dengan built-in functions.
- Fungsi ini memungkinkan pengguna untuk mengembangkan kode yang menerapkan rumus atau persamaan untuk menyelesaikan tugas dan mengembalikan hasil.

1.12 Proses Pembuatan User-Defined Functions

- Untuk membuat fungsi di MySQL, gunakan perintah CREATE FUNCTION diikuti dengan klausa RETURNS dan perintah RETURN.
- Sintaks dasar untuk membuat fungsi:

```
CREATE FUNCTION function_name (parameters)
    RETURNS data_type DETERMINISTIC
BEGIN
    -- logic here
    RETURN value;
END;
```

1.13 Contoh Penggunaan

- Contoh fungsi yang dapat dibuat untuk Lucky Shrub adalah `find_total_cost`, yang menghitung total biaya setelah diskon 10%.

- Sintaks untuk fungsi ini:

```
CREATE FUNCTION find_total_cost(cost DECIMAL)
  RETURNS DECIMAL(5,2) DETERMINISTIC
  BEGIN
    RETURN cost * 0.90;
    -- Menghitung total setelah diskon 10%
  END;
```

- Apa yang Terjadi Tanpa DETERMINISTIC?
 - Hasil yang Tidak Dapat Diprediksi: Meskipun dalam kasus ini fungsi `find_total_cost` selalu mengembalikan hasil yang sama untuk input yang sama (yaitu, menghitung total setelah diskon 10%), tanpa menandai fungsi sebagai DETERMINISTIC, MySQL tidak dapat menganggapnya sebagai fungsi yang konsisten. Ini berarti bahwa setiap kali Anda memanggil fungsi ini, MySQL tidak dapat melakukan optimisasi yang biasanya dilakukan untuk fungsi deterministic.
 - Kinerja yang Lebih Lambat: Tanpa DETERMINISTIC, MySQL harus menghitung hasil fungsi setiap kali dipanggil, bahkan jika inputnya sama. Ini dapat memperlambat kinerja, terutama jika fungsi ini dipanggil berulang kali dalam query yang besar atau kompleks.
 - Penggunaan dalam Indeks: Jika Anda ingin menggunakan fungsi ini dalam indeks atau dalam kondisi tertentu (seperti dalam WHERE clause), MySQL mungkin tidak mengizinkannya karena tidak ada jaminan bahwa hasilnya akan konsisten. Ini dapat membatasi fleksibilitas dalam penggunaan fungsi dalam query yang lebih kompleks.

1.14 Fungsi yang Lebih Kompleks

- Untuk diskon yang berbeda berdasarkan jumlah pembelian, gunakan pernyataan IF ELSE dalam fungsi.
- Sintaks untuk fungsi yang lebih kompleks:

```
CREATE FUNCTION GetTotalCost(cost DECIMAL)
  RETURNS DECIMAL(5,2) DETERMINISTIC
  BEGIN
    DECLARE final_cost DECIMAL(5,2);
    IF cost >= 500 THEN
      SET final_cost = cost * 0.80;
      -- Diskon 20%
    ELSEIF cost >= 100 THEN
      SET final_cost = cost * 0.90;
      -- Diskon 10%
    ELSE
      SET final_cost = cost;
      -- Tanpa diskon
    END IF;
    RETURN final_cost;
  END;
```



```
END IF;  
RETURN final_cost;  
END;
```

1.15 Pengujian Fungsi

Untuk menguji fungsi, gunakan perintah SELECT:

```
SELECT GetTotalCost(500);  
-- Menghasilkan 400 setelah diskon 20%
```

1.16 Menghapus Fungsi

Untuk menghapus fungsi, gunakan perintah:

```
DROP FUNCTION function_name;
```

1.17 Hubungan DETERMINISTIC dengan Memori dan Kinerja

- **Pengelolaan Memori:** Ketika Anda menandai fungsi sebagai DETERMINISTIC, MySQL dapat menyimpan hasil dari fungsi tersebut dalam cache. Ini berarti bahwa jika fungsi dipanggil lagi dengan input yang sama, MySQL dapat mengambil hasil dari cache alih-alih menghitungnya ulang. Ini menghemat penggunaan memori dan mempercepat waktu eksekusi.
- **Optimisasi Kinerja:** Dengan mengetahui bahwa fungsi tersebut selalu memberikan hasil yang sama untuk input yang sama, MySQL dapat melakukan optimisasi lebih lanjut dalam eksekusi query. Ini termasuk menghindari perhitungan yang tidak perlu dan mengurangi beban pada CPU, yang pada gilirannya dapat meningkatkan kinerja keseluruhan database.
- **Penggunaan dalam Indeks:** Fungsi yang ditandai sebagai DETERMINISTIC dapat digunakan dalam indeks dan kondisi query lainnya. Ini memungkinkan Anda untuk memanfaatkan fungsi tersebut dalam cara yang lebih fleksibel dan efisien, yang tidak mungkin dilakukan jika fungsi tidak deterministic.

1.18 Apa itu Delimiter?

Dalam MySQL, DELIMITER adalah karakter atau string yang digunakan untuk menandai akhir dari sebuah pernyataan SQL. Secara default, delimiter di MySQL adalah titik koma (;). Namun, ketika Anda membuat fungsi atau prosedur yang mengandung beberapa pernyataan SQL, Anda perlu mengubah delimiter untuk menghindari kebingungan antara akhir dari pernyataan SQL dan akhir dari fungsi atau prosedur itu sendiri.

1.19 Mengapa Mengubah Delimiter Penting?

- Menghindari Kesalahan Sintaks: Ketika Anda menulis fungsi atau prosedur yang memiliki beberapa pernyataan, jika Anda tidak mengubah delimiter, MySQL akan menganggap setiap titik koma sebagai akhir dari pernyataan. Ini dapat menyebabkan kesalahan sintaks dan membuat MySQL tidak dapat memahami kode Anda.
- Memudahkan Penulisan Kode: Dengan mengubah delimiter, Anda dapat menulis kode yang lebih kompleks tanpa harus khawatir tentang kesalahan yang disebabkan oleh titik koma yang tidak diinginkan.

1.20 Cara Mengubah Delimiter

Berikut adalah langkah-langkah untuk mengubah delimiter saat membuat fungsi atau prosedur:

1. Ubah Delimiter: Gunakan perintah **DELIMITER** untuk mengubah delimiter dari titik koma (;) ke karakter lain, seperti // atau \$\$.

```
DELIMITER //
```

2. Tulis Fungsi atau Prosedur: Setelah mengubah delimiter, Anda dapat menulis fungsi atau prosedur Anda.

```
CREATE FUNCTION example_function()  
  RETURNS INT  
  BEGIN  
    DECLARE result INT;  
    SET result = 1 + 1;  
    RETURN result;  
  END //
```

3. Kembalikan Delimiter ke Default: Setelah selesai menulis fungsi atau prosedur, jangan lupa untuk mengembalikan delimiter ke default (;).

```
DELIMITER ;
```

1.21 Contoh Penggunaan DELIMITER

Berikut adalah contoh lengkap yang menunjukkan penggunaan delimiter saat membuat fungsi:

```
DELIMITER //  
  
CREATE FUNCTION calculate_discount(price DECIMAL)  
  RETURNS DECIMAL(5,2)  
  BEGIN
```

```
RETURN price * 0.90;
-- Menghitung total setelah diskon 10%
END //

DELIMITER ;
```

1.22 Membuat Stored Procedure

- Anda perlu menggunakan perintah delimiter untuk mengubah delimiter dari default semicolon (;) menjadi double forward slash (//) agar MySQL dapat mengkompilasi kode dalam blok begin-end sebagai satu pernyataan komposit.
- Setelah membuat prosedur, ubah delimiter kembali ke semicolon (;) untuk melanjutkan penggunaan MySQL seperti biasa.

1.23 Perbedaan Antara Fungsi dan Prosedur

- Pengembalian Nilai:
Fungsi mengembalikan satu nilai, sedangkan prosedur dapat mengembalikan satu nilai, beberapa nilai, atau tidak mengembalikan nilai sama sekali.
- Parameter:
Fungsi hanya menerima parameter input, sementara prosedur dapat menerima parameter IN, OUT, dan INOUT.
- Penggunaan:
Fungsi biasanya digunakan untuk mengenkapsulasi rumus umum atau aturan bisnis yang dapat digunakan kembali dalam pernyataan SQL dan prosedur tersimpan. Prosedur lebih sering digunakan untuk memproses, memanipulasi, dan mengubah data dalam basis data.
- Cara Memanggil:
Fungsi dapat dipanggil dari mana saja, termasuk dalam pernyataan SELECT dan prosedur tersimpan. Prosedur hanya dapat dipanggil menggunakan pernyataan CALL.
- Pembuatan:
Fungsi dibuat menggunakan pernyataan CREATE FUNCTION, sedangkan prosedur dibuat menggunakan pernyataan CREATE PROCEDURE.
- Tipe Data return:
Fungsi harus menentukan tipe data dari nilai yang dikembalikan dalam pernyataan RETURNS, sedangkan prosedur tidak perlu melakukan ini.

1.24 Apa itu MySQL Trigger

- TRIGGER adalah sekumpulan tindakan yang tersedia dalam bentuk program tersimpan (stored program) yang diaktifkan secara otomatis ketika peristiwa tertentu terjadi, seperti saat data diinsert, diupdate, atau dihapus dari tabel.
- Sebelum menggunakan trigger, Anda perlu membuatnya dan mungkin juga perlu menghapusnya setelah tidak diperlukan lagi.

1.25 Cara Membuat dan Menghapus Trigger

- Membuat Trigger: Sintaks untuk membuat trigger adalah:

```
CREATE TRIGGER trigger_name
[BEFORE | AFTER] [INSERT | UPDATE | DELETE]
ON table_name
FOR EACH ROW
BEGIN
    -- logic here
END;
```

- Anda harus memberikan nama unik untuk trigger dan menentukan jenis trigger (insert, update, delete) serta kapan trigger tersebut dieksekusi (sebelum atau sesudah peristiwa).
- Menghapus Trigger: Sintaks untuk menghapus trigger adalah:

```
DROP TRIGGER IF EXISTS schema_name.trigger_name;
```

- Pastikan untuk menggunakan klausa IF EXISTS agar perintah hanya berfungsi jika MySQL dapat menemukan trigger tersebut.

1.26 Contoh Penggunaan Trigger

- Dalam konteks tim penjualan Lucky Shrub, mereka perlu menambahkan trigger yang menandai item ketika diskon yang diberikan melebihi 25%.
- Trigger dapat dibuat dengan perintah CREATE TRIGGER, memberi nama trigger sebagai "approval_request", dan menetapkan jenis trigger sebagai "AFTER UPDATE".

1.27 Manfaat Trigger

- Trigger berguna untuk menjaga log catatan atau perubahan yang dilakukan dalam database, berfungsi sebagai cara untuk mempertahankan jejak audit.

- Mereka juga dapat membantu menjaga integritas data dengan memastikan semua data diperbarui sesuai kebutuhan dan melakukan tugas secara otomatis berdasarkan tindakan tertentu pada tabel database.

1.28 Jenis-Jenis Triggers

- Row Level Triggers: Diaktifkan untuk setiap baris yang dimasukkan, diperbarui, atau dihapus dalam tabel. Misalnya, jika 100 baris ditambahkan, trigger ini akan diaktifkan 100 kali.
- Statement Level Triggers: Diaktifkan sekali untuk setiap tindakan, terlepas dari jumlah baris yang terlibat. Contohnya, satu pernyataan INSERT yang menambahkan 100 baris hanya mengaktifkan trigger ini sekali.

1.29 Klasifikasi Trigger

- Before Triggers: Diaktifkan sebelum tindakan dilakukan pada tabel.
- After Triggers: Diaktifkan setelah tindakan dilakukan pada tabel.

1.30 Contoh Trigger

- Before Insert Trigger: Diaktifkan sebelum peristiwa INSERT terjadi.
- After Insert Trigger: Diaktifkan setelah peristiwa INSERT terjadi.
- Before Update Trigger: Diaktifkan sebelum peristiwa UPDATE terjadi.
- After Update Trigger: Diaktifkan setelah peristiwa UPDATE terjadi.
- Before Delete Trigger: Diaktifkan sebelum data dihapus dari tabel.
- After Delete Trigger: Diaktifkan setelah data dihapus dari tabel.

1.31 Sintaks untuk Membuat Trigger

Sintaks untuk membuat trigger adalah sebagai berikut:

```
CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- logic here
END;
```

1.32 Contoh Penerapan Trigger

Contoh penerapan trigger untuk perusahaan Lucky Shrub:

- Mereka ingin memastikan bahwa tidak ada nilai negatif yang dapat dimasukkan ke dalam kolom order quantity. Mereka dapat membuat trigger sebagai berikut:

```
CREATE TRIGGER order_quantity_check
BEFORE INSERT ON orders FOR EACH ROW
BEGIN
    IF NEW.order_quantity < 0 THEN
        SET NEW.order_quantity = 0;
    END IF;
END;
```

- After Insert Trigger: Untuk mencatat setiap kali pesanan baru dimasukkan ke dalam tabel audit.
- After Delete Trigger: Untuk mencatat tanggal dan waktu ketika catatan pesanan dihapus dari tabel.

1.33 Membuat Trigger

- TRIGGER adalah sekumpulan aksi yang secara otomatis diaktifkan saat terjadi peristiwa seperti INSERT, UPDATE, atau DELETE.
- Sintaks untuk Membuat Trigger:

```
CREATE TRIGGER trigger_name
BEFORE|AFTER INSERT|UPDATE|DELETE ON table_name FOR EACH
ROW
BEGIN
    -- logika di sini
END;
```

- Contoh Kasus: Lucky Shrub ingin memastikan tidak ada nilai negatif yang dimasukkan ke dalam kolom quantity saat order baru dicatat. Jika nilai negatif terdeteksi, maka harus diatur ke nilai default 0 menggunakan trigger before-insert.

1.34 Menghapus Trigger

- Sintaks untuk Menghapus Trigger:

```
DROP TRIGGER IF EXISTS schema_name.trigger_name;
```

- Pentingnya Menghapus Trigger: Jika Anda menghapus tabel yang terkait dengan trigger, semua trigger yang terkait juga akan dihapus.

1.35 Pengertian MySQL Scheduled Events

- Scheduled Event: Tugas yang dieksekusi sesuai dengan jadwal tertentu. Setiap event memiliki nama unik dan berisi satu atau lebih pernyataan SQL.
- Tipe Event: Ada dua jenis utama:
 - One Time Events: Terjadi hanya sekali, misalnya, memasukkan data ke dalam tabel satu jam dari sekarang.
 - Recurring Events: Terjadi secara berkala, seperti menghasilkan laporan mingguan dari database.

1.36 Sintaks untuk Membuat Scheduled Event

- Untuk membuat event, gunakan kata kunci CREATE EVENT. Sintaksnya adalah:

```
CREATE EVENT event_name ON SCHEDULE schedule
DO
BEGIN
    -- logic here
END;
```

- Untuk event satu kali, gunakan klausa AT diikuti dengan timestamp. Contoh:

```
CREATE EVENT GenerateRevenueReport ON SCHEDULE AT
CURRENT_TIMESTAMP + INTERVAL 12 HOUR
DO
BEGIN
    -- logic untuk menghasilkan laporan
END;
```

- Untuk event berulang, gunakan klausa EVERY. Contoh:

```
CREATE EVENT daily_restock ON SCHEDULE EVERY 1 DAY
DO
BEGIN
    -- logic untuk memeriksa dan memperbarui stok
END;

CREATE EVENT IF NOT EXISTS daily_restock
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP() + INTERVAL 20 DAY
ENDS CURRENT_TIMESTAMP() + INTERVAL 40 DAY
DO
BEGIN
```

```
-- logic untuk memeriksa dan memperbarui stok  
END;
```

1.37 Menghapus Scheduled Event

Untuk menghapus event yang tidak lagi diperlukan, gunakan pernyataan DROP EVENT. Sintaksnya adalah:

```
DROP EVENT IF EXISTS event_name;
```

1.38 MySQL Triggers dan Penggunaannya

- Definisi Trigger: MySQL trigger adalah program yang disimpan dan terkait dengan tabel. Trigger diaktifkan sebelum atau setelah operasi SQL tertentu dilakukan pada tabel, seperti INSERT, UPDATE, dan DELETE.
- Row-Level Triggers: Trigger ini diaktifkan sebelum atau setelah baris dimasukkan, diperbarui, atau dihapus dalam tabel. Anda dapat menentukan serangkaian tindakan yang akan dilakukan pada tabel saat operasi yang ditentukan terjadi.

1.39 Membuat dan Menghapus MySQL Triggers

Sintaks untuk Membuat Trigger:

```
CREATE TRIGGER trigger_name {BEFORE | AFTER} {INSERT |  
UPDATE | DELETE} ON table_name FOR EACH ROW trigger_body;
```

- Trigger Name: Nama unik untuk trigger dalam skema atau database.
- Trigger Time: Menentukan kapan trigger diaktifkan (BEFORE atau AFTER).
- Trigger Event: Jenis operasi yang mengaktifkan trigger (INSERT, UPDATE, DELETE).
- Table Name: Tabel yang terkait dengan trigger.
- Trigger Body: Pernyataan yang dieksekusi saat trigger diaktifkan.

Sintaks untuk Menghapus Trigger:

```
DROP TRIGGER [IF EXISTS] [schema_name.] trigger_name;
```


1.40 Mengakses Kolom Tabel dalam Trigger

OLD dan NEW Modifiers: Digunakan untuk mengakses nilai kolom sebelum dan setelah operasi.

- OLD: Nilai kolom sebelum operasi (misalnya, sebelum INSERT, UPDATE, DELETE).
- NEW: Nilai kolom setelah operasi.

1.41 Keuntungan dan Kerugian dari Triggers

Keuntungan:

- Memungkinkan validasi data sebelum dimasukkan atau diperbarui.
- Menyediakan cara alternatif untuk menjalankan tugas terjadwal atau melakukan tugas secara otomatis.
- Meningkatkan kinerja kueri SQL karena tidak perlu dikompilasi setiap kali kueri dieksekusi.
- Mengurangi kode sisi klien untuk validasi, sehingga menghemat waktu dan usaha.

Kerugian:

- Tidak semua jenis validasi batasan dapat dilakukan menggunakan trigger.
- Sulit untuk melakukan troubleshooting karena bekerja di lapisan database.
- Dapat meningkatkan beban pada server database.

2 Database Optimization

2.1 Konsep Database Optimization

- Database optimization adalah proses meningkatkan kinerja sistem database untuk mengurangi waktu yang dibutuhkan untuk menjalankan query, memproses, dan mengirimkan hasil query kepada pengguna.
- Pentingnya: Dengan database yang teroptimasi, query SQL dapat diproses dan data yang diperlukan dapat dikembalikan dengan cepat.

2.2 Jenis Pernyataan SQL

- Data Retrieval Statements: Pernyataan ini mengembalikan data dari database, dikenal juga sebagai SELECT statements.
- Data Change Statements: Digunakan untuk mengubah data dalam database, seperti INSERT, UPDATE, dan DELETE.

2.3 Teknik Optimasi untuk Data Retrieval Statemments

- Indexes: Digunakan untuk mempercepat pencarian data. Indexes dibuat pada kolom tabel.
- Optimasi SELECT Statements:
 - Menargetkan kolom tertentu dalam perintah SELECT.
 - Menggunakan fungsi dan wildcard secara efisien.
 - Memanfaatkan INNER JOIN daripada OUTER JOIN.
 - Menggunakan klausa DISTINCT dan UNION.
 - Pentingnya menggunakan klausa ORDER BY untuk mengurutkan hasil.

2.4 Teknik Optimasi untuk Data Change Statements

- UPDATE dan DELETE Statements: Optimasi dimulai dengan mengoptimalkan kondisi dalam klausa WHERE.
- INSERT Statements: Dapat dioptimasi dengan melakukan batch inserts, yaitu menyisipkan lebih dari satu baris dalam satu operasi insert.

2.5 Pentingnya Optimasi Kueri SELECT

- Kueri SQL yang tidak dioptimalkan dapat menambah beban pada database dan memperlambat kinerjanya.
- Optimasi diperlukan agar database dapat mengeksekusi kueri dengan cepat dan efisien.

2.6 Pedoman Dasar untuk Optimasi SELECT

- Targetkan hanya kolom yang diperlukan dalam klausa SELECT. Hindari menggunakan * jika tidak membutuhkan semua kolom.
- Hindari penggunaan fungsi dalam predikat dan penggunaan wildcard di awal dalam predikat. Predikat adalah istilah untuk ekspresi yang mengembalikan nilai true/false, biasanya dipakai di klausa WHERE. **Wildcard** ada pada keyword LIKE, yaitu simbol %. Hindari penggunaan wildcard di awal pattern-nya jika memungkinkan.
- Gunakan INNER JOIN jika memungkinkan, dan gunakan DISTINCT serta UNION hanya saat diperlukan.

2.7 Contoh Praktis Optimasi SELECT

- Menggunakan asterisk (*) dalam pernyataan SELECT untuk mengambil semua data dapat memperlambat kinerja. Sebaiknya, sebutkan hanya kolom yang diperlukan.
- Hindari menggunakan fungsi dalam klausa WHERE pada kolom yang diindeks, karena ini mencegah database menggunakan indeks.
- Penggunaan wildcard di awal dalam predikat (misalnya, menggunakan LIKE dengan wildcard di depan) juga dapat memperlambat pencarian.
- INNER JOIN lebih efisien dibandingkan OUTER JOIN karena hanya mengambil data yang diperlukan.
- Menggunakan UNION ALL daripada UNION dapat mempercepat proses eksekusi dengan menghindari operasi pengurutan.

2.8 Mengoptimalkan Kueri SELECT Pada WHERE

- Lucky Shrub perlu menemukan pesanan yang tiba pada tanggal 12 September. Mereka dapat menggunakan kueri SELECT dengan klausa WHERE dan fungsi DATE_ADD, tetapi ini memberikan beban tambahan pada database.
- Solusi yang lebih efisien adalah dengan membuat kolom kustom di tabel pesanan yang disebut **expected_delivery_date**, yang menunjukkan tanggal pengiriman yang diharapkan. Dengan cara ini, mereka hanya perlu memindai kolom ini untuk nilai yang cocok dengan 12 September.
- Ini untuk menghindari pemakaian fungsi di klausa WHERE.

2.9 Menggunakan Wildcard dan Indeks

- Untuk memproses pesanan dari pelanggan bernama Ito, mereka perlu menemukan detail pelanggan di tabel klien. Salah satu metode adalah menggunakan kueri SELECT dengan wildcard di depan dan operator LIKE, tetapi MySQL tidak dapat menggunakan indeks dengan wildcard di depan.
- Solusinya adalah menambahkan kolom baru ke tabel klien menggunakan pernyataan ALTER TABLE dan menyebutnya `reverse_full_name`, yang berisi nama klien yang dibalik. Setelah itu, mereka dapat membuat indeks pada kolom baru ini. Dengan cara ini, mereka dapat menggunakan wildcard di belakang dengan operator LIKE dan tetap menggunakan indeks.

2.10 Mengambil Data dari Tabel dengan Join

- Departemen keuangan perlu melaporkan semua pesanan yang dilakukan. Mereka dapat mengekstrak informasi ini dengan menargetkan tabel produk dan pesanan di database.
- Biasanya, ini dapat dilakukan dengan menggunakan kueri OUTER JOIN, tetapi kueri ini juga mengembalikan catatan yang tidak cocok dari kedua tabel. Metode yang lebih efisien adalah menggunakan INNER JOIN yang menargetkan kolom `product_id` yang sama dari kedua tabel, sehingga hanya catatan yang cocok yang dikembalikan.
- LEFT JOIN dan RIGHT JOIN termasuk OUTER JOIN.

2.11 Pentingnya Optimasi Kueri SELECT

- Mengoptimalkan kueri SELECT melibatkan pengurangan waktu yang diperlukan untuk mengambil data dari database.
- Kueri SELECT, yang juga dikenal sebagai pernyataan pengambilan data, sangat penting dalam aplikasi database karena melakukan semua operasi pencarian database.

2.12 Teknik untuk Mengoptimalkan Kueri SELECT

Buat indeks pada kolom tabel yang sering di-query untuk mengurangi jumlah baris yang perlu dibaca atau dipindai.

Beberapa teknik yang dapat membantu MySQL Query Optimizer meliputi:

- Menghindari penggunaan kolom yang tidak perlu dalam klausa SELECT.
- Menghindari penggunaan fungsi dalam predikat (kondisi klausa WHERE).
- Menghindari penggunaan wildcard di depan dalam predikat, terutama dengan operator LIKE.

- Menggunakan INNER JOIN daripada OUTER JOIN.
- Menggunakan DISTINCT dan UNION dengan hemat dalam kueri SELECT.

2.13 Mengoptimalkan Kueri SELECT dengan Pernyataan Explain

Ketika kueri SELECT dijalankan, MySQL Query Optimizer membuat rencana eksekusi kueri yang optimal. Anda dapat melihat rencana ini dengan menambahkan EXPLAIN di depan kueri.

Sintaks untuk menggunakan EXPLAIN adalah sebagai berikut:

```
EXPLAIN SELECT column_name FROM table_name WHERE VALUE;
```

2.14 Contoh Hasil Pernyataan EXPLAIN

Dalam contoh database Lucky Shrub, mereka menggunakan kueri berikut untuk mendapatkan nomor kontak klien Jane Delgado:

```
EXPLAIN SELECT ContactNumber FROM Clients WHERE FullName='Jane_Delgado';
```

Hasil dari EXPLAIN memberikan informasi tentang bagaimana kueri dieksekusi, termasuk kolom-kolom penting seperti ID, SELECT_TYPE, Table, dan lainnya.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Clients	NULL	ALL	NULL	NULL	NULL	NULL	10	10.00	Using where

Figure 1:

2.15 Penjelasan Kolom dalam Hasil EXPLAIN

- ID:
Ini adalah pengidentifikasi urutan untuk setiap pernyataan SELECT dalam kueri. Nilai ini lebih berarti ketika ada kueri bersarang (*subqueries*). Dalam contoh Lucky Shrub, nilai ini adalah 1 karena ini adalah kueri sederhana.
- SELECT_TYPE:
Menampilkan jenis kueri SELECT yang akan dieksekusi. Beberapa nilai yang mungkin termasuk:
 - SIMPLE: Kueri SELECT sederhana tanpa subquery atau UNION.
 - PRIMARY: Kueri SELECT berada di luar kueri JOIN.

- DERIVED: Kueri SELECT adalah bagian dari subquery dalam klausa FROM.
 - SUBQUERY: Kueri SELECT pertama dalam subquery.
 - DEPENDENT SUBQUERY: Kueri SELECT yang bergantung pada kueri luar.
 - UNION: Kueri SELECT adalah pernyataan kedua atau lebih dari UNION.
 - DEPENDENT UNION: Kueri UNION yang bergantung pada kueri luar.
 - UNION RESULT: Kueri SELECT adalah hasil dari UNION.
- Table:
Menampilkan nama tabel yang dirujuk dalam kueri SELECT. Dalam contoh Lucky Shrub, ini adalah tabel Clients.
 - Partitions:
Menampilkan partisi tempat data berada. Partisi memungkinkan distribusi sebagian data tabel di seluruh sistem file. Jika kueri hanya mengakses sebagian kecil data tabel, maka lebih sedikit catatan yang perlu dipindai, sehingga kueri dapat dieksekusi lebih cepat. Namun, ini lebih berarti saat berurusan dengan set data besar. Dalam contoh ini, tabel Clients tidak dipartisi, sehingga menunjukkan nilai NULL.
 - type: Menunjukkan cara pencarian dilakukan. Beberapa nilai yang mungkin termasuk:
 - system: Tabel hanya memiliki satu baris atau tidak ada baris.
 - const: Nilai kolom yang dicari dapat dianggap sebagai konstanta (hanya ada satu baris yang cocok).
 - eq_ref: Indeks terkluster digunakan oleh operasi (indeks adalah PRIMARY KEY atau UNIQUE INDEX).
 - ref: Kolom yang diindeks diakses menggunakan operator kesetaraan.
 - fulltext: Pemindaian menggunakan indeks FULLTEXT tabel.
 - index: Seluruh indeks dipindai untuk menemukan kecocokan.
 - all: Seluruh tabel dipindai untuk menemukan baris yang cocok. Ini adalah jenis pemindaian terburuk dan biasanya menunjukkan kurangnya indeks yang sesuai.
 - possible_keys:
Menunjukkan kunci yang dapat digunakan oleh MySQL untuk menemukan baris dari tabel. Jika nilai kolom ini NULL, maka tidak ada indeks relevan yang ditemukan.

- **key:**
Menunjukkan indeks yang sebenarnya digunakan oleh MySQL. Jika nilai kolom ini NULL, berarti tidak ada indeks dalam tabel yang dapat digunakan oleh optimizer.
- **key_len:**
Menunjukkan panjang indeks yang dipilih oleh Query Optimizer. Misalnya, nilai **key_len** 4 berarti memerlukan memori untuk menyimpan empat karakter. Jika NULL, berarti tidak ada indeks yang digunakan.
- **ref:**
Menunjukkan kolom tabel mana yang dibandingkan dengan indeks untuk melakukan pencarian. Nilai const berarti konstanta, sementara func berarti nilai yang digunakan berasal dari fungsi. Jika NULL, tidak ada kolom atau konstanta yang dibandingkan dengan indeks.
- **rows:**
Menampilkan jumlah catatan yang diperiksa untuk menghasilkan output. Dalam contoh Lucky Shrub, hasil menunjukkan bahwa 10 catatan diperiksa, yang berarti setiap baris dalam tabel diperiksa. Ini menunjukkan penggunaan sumber daya database yang tidak efisien.
- **filtered:**
Menunjukkan persentase perkiraan dari jumlah baris tabel yang telah difilter oleh kondisi tertentu. Semakin tinggi persentasenya, semakin baik kinerja kueri.
- **Extra:**
Berisi informasi tambahan mengenai rencana eksekusi kueri. Perhatikan jika ada nilai seperti "Using temporary" atau "Using filesort" karena ini menunjukkan kueri yang bermasalah. Contoh nilai yang mungkin:
 - Using temporary: Menunjukkan bahwa MySQL perlu membuat tabel sementara untuk menyimpan hasil kueri.
 - Using filesort: Menunjukkan bahwa MySQL harus melakukan pass tambahan untuk menentukan cara mengambil baris dalam urutan yang terurut.

2.16 Rencana Kueri SELECT yang Lebih Efisien

Menggunakan indeks pada kolom yang diperlukan menghasilkan kueri SELECT yang lebih efisien.

Contoh untuk menambahkan indeks pada kolom FullName:

```
CREATE INDEX IdxFullName ON Clients(FullName);
```

Sehingga hasil dari EXPLAIN akan berubah,

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Clients	NULL	ref	IdxFullName	IdxFullName	803	const	1	100.00	NULL

Figure 2:

2.17 Pengertian Index

- Index adalah struktur data yang membantu mempertahankan pointer yang mengarah ke data yang terurut. Meskipun tidak terlihat dalam database, index dapat dibayangkan sebagai tabel dengan dua kolom: satu untuk pointer dan satu untuk data yang terurut.
- Indeks dibuat pada kolom tabel. Tidak semua kolom perlu diindeks; pilih kolom yang sering digunakan untuk memfilter query SELECT. Data kolom yang diindeks disimpan di lokasi terpisah yang disebut indeks, dan data diurutkan berdasarkan tipe (misalnya numerik, alfabetis, atau tanggal).

2.18 Jenis-jenis Index

- Primary Index: Juga dikenal sebagai clustered index, disimpan dalam tabel itu sendiri dan dihasilkan secara otomatis saat tabel dibuat dengan primary atau unique key.
- Secondary Index: Diciptakan menggunakan pernyataan CREATE INDEX. Sintaksnya adalah:

```
CREATE INDEX idx_column_name ON table_name (column_name)
;
```

- Index ini dapat dibuat menggunakan satu atau lebih kolom dari tabel.

2.19 Operasi Query dan Indeks di MySQL

- Filtering rows menggunakan klausa WHERE:
Indeks digunakan untuk mempercepat pencarian baris yang memenuhi kondisi tertentu. Jika ada beberapa indeks pada kolom yang sama, MySQL akan memilih indeks yang dapat memproses jumlah baris minimum, sehingga mengurangi waktu eksekusi.
- Performing JOIN operations:
Indeks dapat digunakan untuk meningkatkan kinerja saat melakukan operasi JOIN. Untuk mendapatkan kinerja terbaik dari indeks, kolom yang digunakan dalam JOIN harus memiliki tipe data yang sama. Ini membantu MySQL dalam mengoptimalkan pencarian data yang relevan dari tabel yang berbeda.

- MIN dan MAX functions:

Indeks juga dapat digunakan dengan fungsi MIN dan MAX untuk menemukan nilai minimum dan maksimum dari kolom tertentu. Ini memungkinkan query untuk lebih cepat dalam mendapatkan hasil yang diinginkan tanpa memindai seluruh tabel.

- Sorting and grouping operations:

Indeks dapat digunakan dalam operasi pengurutan (ORDER BY) dan pengelompokan (GROUP BY). Dengan adanya indeks, MySQL dapat mengakses data yang sudah terurut, sehingga mempercepat proses pengurutan dan pengelompokan.

2.20 Proses Penggunaan Index

- Contoh Kasus: Lucky Shrub perlu mengambil nomor kontak klien dari database yang memiliki ribuan entri. Dengan menggunakan secondary index pada kolom nama lengkap, proses pencarian dapat dilakukan lebih cepat.

- Langkah-Langkah:

1. Gunakan pernyataan EXPLAIN untuk melihat bagaimana database mengeksekusi query.
2. Lakukan query SELECT untuk mengambil nomor kontak yang sesuai.
3. Jika tidak ada index, MySQL harus memindai semua baris untuk menemukan nilai yang cocok.
4. Buat secondary index dengan sintaks:

```
CREATE INDEX idx_full_name ON
clients (full_name);
```

5. Uji efisiensi index dengan pernyataan EXPLAIN lagi untuk melihat pengurangan jumlah baris yang dipindai.

2.21 Definisi Transaksi

Transaction adalah sekumpulan satu atau lebih query yang dapat disimpan secara permanen ke dalam database. Jika ada kesalahan, database dapat dikembalikan/rollback ke keadaan semula.

2.22 Pernyataan Transaksi dalam MySQL

- START TRANSACTION: Memulai proses transaksi. Ini menandai titik yang akan dikembalikan jika Anda memutuskan untuk melakukan rollback.

```
START TRANSACTION;
```

- COMMIT: Menyimpan perubahan yang telah dilakukan selama transaksi ke dalam database secara permanen.

```
COMMIT ;
```

- ROLLBACK: Mengembalikan database ke keadaan semula jika terjadi kesalahan selama transaksi. Harus dilakukan sebelum COMMIT.

```
ROLLBACK ;
```

2.23 Proses Pengelolaan Transaksi

- Memulai Transaksi: Gunakan START TRANSACTION untuk memulai.
- Menulis Query: Tambahkan query SQL yang diperlukan, seperti INSERT dan UPDATE.
- Menggunakan ROLLBACK: Jika terjadi kesalahan, gunakan ROLLBACK untuk mengembalikan perubahan.
- Menyimpan Perubahan: Jika semua query berhasil, gunakan COMMIT untuk menyimpan perubahan.

2.24 Contoh Kasus Transaksi

Lucky Shrub melakukan pembaruan pada tabel pesanan dan produk. Jika terjadi kesalahan, mereka dapat menggunakan ROLLBACK untuk mengembalikan data ke keadaan semula.